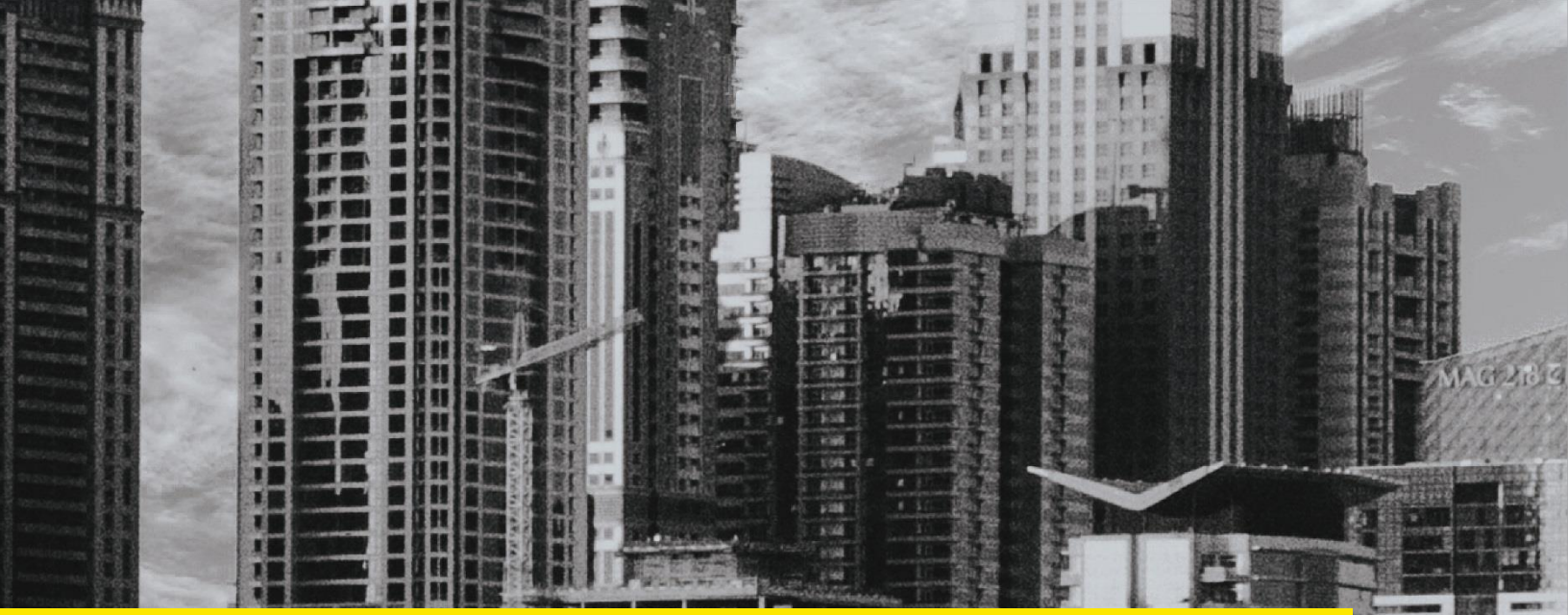


DOCUMENTATION REPORT

Ncedo Masiza:https://github.com/Ncedo-Masiza/banking-system-and-database

Peterson Headger;https://github.com/Peterson-JR557/Banking-System-

Kamohelo Molaetsa:https://github.com/molaetsakamohelo-10/BankingSystem

Assignment title: Banking System Assignment

Lecturer name:MR OWAMI

Submission date:26/05/2026

The Shield

Table of Contents

1 INTRODUCTION

1.1 Purpose and Scope of the System

The purpose of this project is to develop a Java-based Banking Management System that interacts with a relational database. The system is designed to manage customers, accounts, and banking transactions efficiently and securely.

The system allows users to:

- Register customers
- Create bank accounts
- Deposit money
- Withdraw money
- Transfer funds
- Check account balances

1.2 Background and Motivation

Banking systems are important for managing financial operations in modern institutions. Traditional manual systems are slow and prone to errors. This project was developed to automate banking operations and improve transaction management.

The project also provides practical experience in:

- Java programming
- SQL database design
- JDBC integration
- Error handling
- System testing

1.3 Objectives

The objectives of this project are:

- To develop a functional banking application using Java
- To connect the application to a relational database
- To implement customer and account management
- To process banking transactions securely

- To apply object-oriented programming concepts
- To create a normalized database structure

2. System Overview

2.1 High-Level Description

The Banking Management System is a Java application connected to a MySQL database. The system allows users to perform common banking operations through a simple interface.

The system includes the following modules:

- Customer Management
- Account Management
- Transaction Processing
- Balance Enquiry

2.2 Core Functionalities

Customer Management

This module allows users to:

- Register new customers
- View customer details
- Update customer information

Account Management

This module allows users to:

- Create accounts
- Manage account information
- Store account balances

Transaction Processing

This module supports:

- Deposits
- Withdrawals
- Transfers between accounts

Balance Enquiry

This feature allows users to:

- View current account balances

2.3 Technologies Used

Technology	Purpose
Java	Application development
MySQL	Relational database
JDBC	Database connectivity
NetBeans / IntelliJ	IDE
SQL	Database queries

3. DATABASE DESIGN

3.1 ERD (Entity Relationship Diagram)

Customer Table

Field Name	Data Type	Description
customer_id	INT	Primary key
first_name	VARCHAR	Customer first name
last_name	VARCHAR	Customer surname
phone	VARCHAR	Contact number
email	VARCHAR	Email address

Purpose:
The Customer table stores customer personal information.

Account Table

Field Name	Data Type	Description
account_id	INT	Primary key
customer_id	INT	Foreign key
account_type	VARCHAR	Savings/Current
balance	DECIMAL	Current balance

Purpose:
The Account table stores customer bank accounts and balances.

Transaction Table

Field Name	Data Type	Description
transaction_id	INT	Primary key
account_id	INT	Foreign key
transaction_type	VARCHAR	Deposit/Withdrawal
amount	DECIMAL	Transaction amount

Field Name	Data Type	Description
transaction_date	DATETIME	Date and time

Purpose:
The Transaction table stores all banking transactions.

Transfer

Field	Type
transfer_id	INT PK
from_account	INT FK
to_account	INT FK
amount	DECIMAL
transfer_date	DATETIME

3.3 Primary and Foreign Key Justification

Primary Keys

Primary keys uniquely identify records in each table.

Examples:

- customer_id uniquely identifies customers
- account_id uniquely identifies accounts

Foreign Keys

Foreign keys maintain relationships between tables.

Examples:

- customer_id in Account table links accounts to customers
- account_id in Transaction table links transactions to accounts

---

3.4 Normalisation

The database design follows Third Normal Form (3NF).

First Normal Form (1NF)

- No repeating groups

- Atomic values used

Second Normal Form (2NF)

- All non-key attributes depend on the primary key

Third Normal Form (3NF)

- No transitive dependencies
- Redundant data minimized

This improves:

- Data consistency
- Database efficiency
- Reduced duplication

4. APPLICATION FEATURES DOCUMENTATION

4.1 Feature List with Descriptions

Feature 1: Register Customer

Allows users to create a new customer profile.

Feature 2: Create Account

Allows creation of savings or current accounts.

Feature 3: Deposit Money

Allows money to be added into accounts.

Feature 4: Withdraw Money

Allows money withdrawal after balance validation.

Feature 5: Transfer Funds

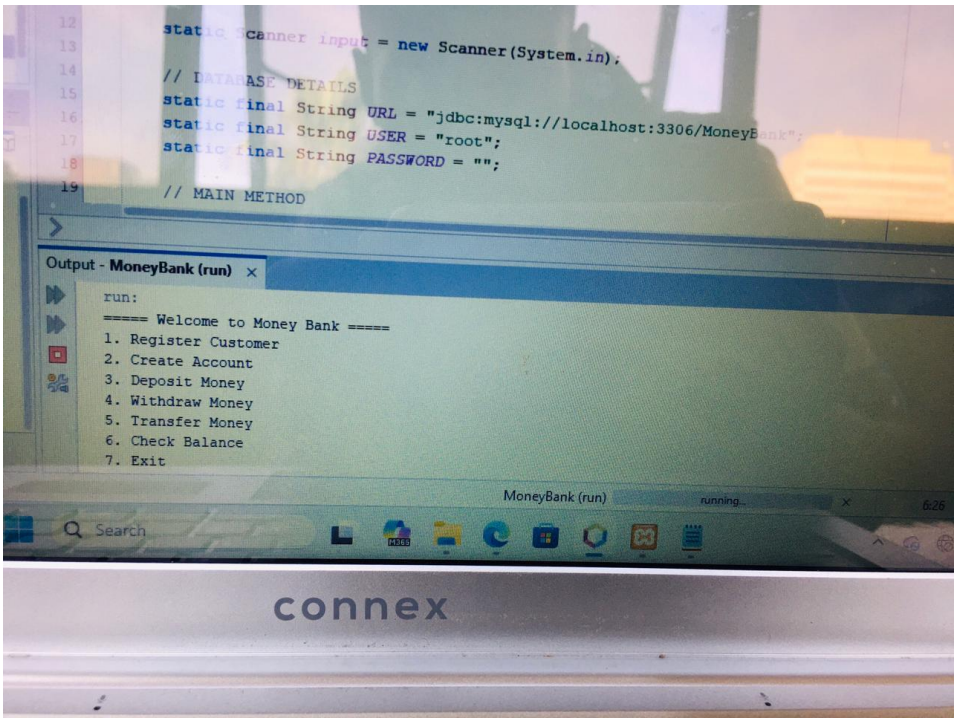
Allows transfers between accounts.

Feature 6: Balance Enquiry

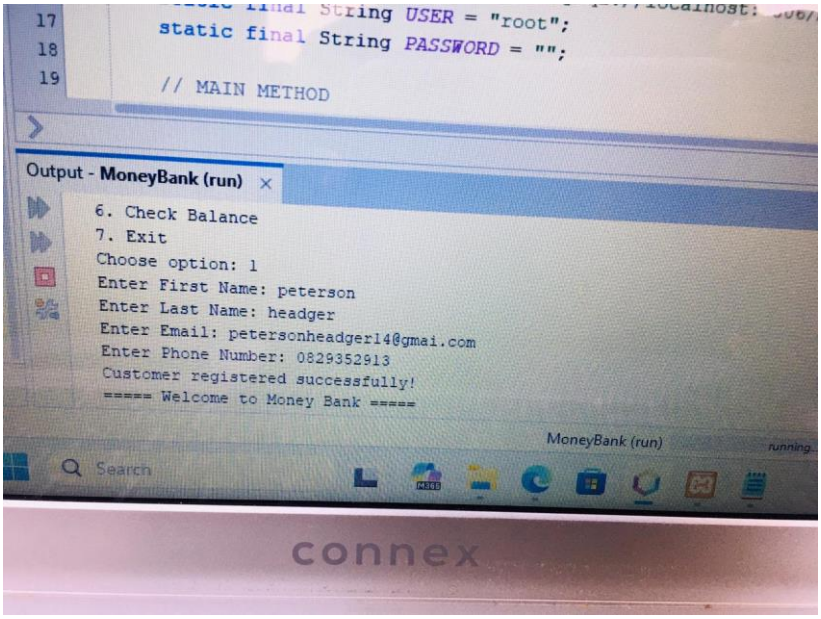
Displays the current account balance.

4.2 Screenshots of Running Application

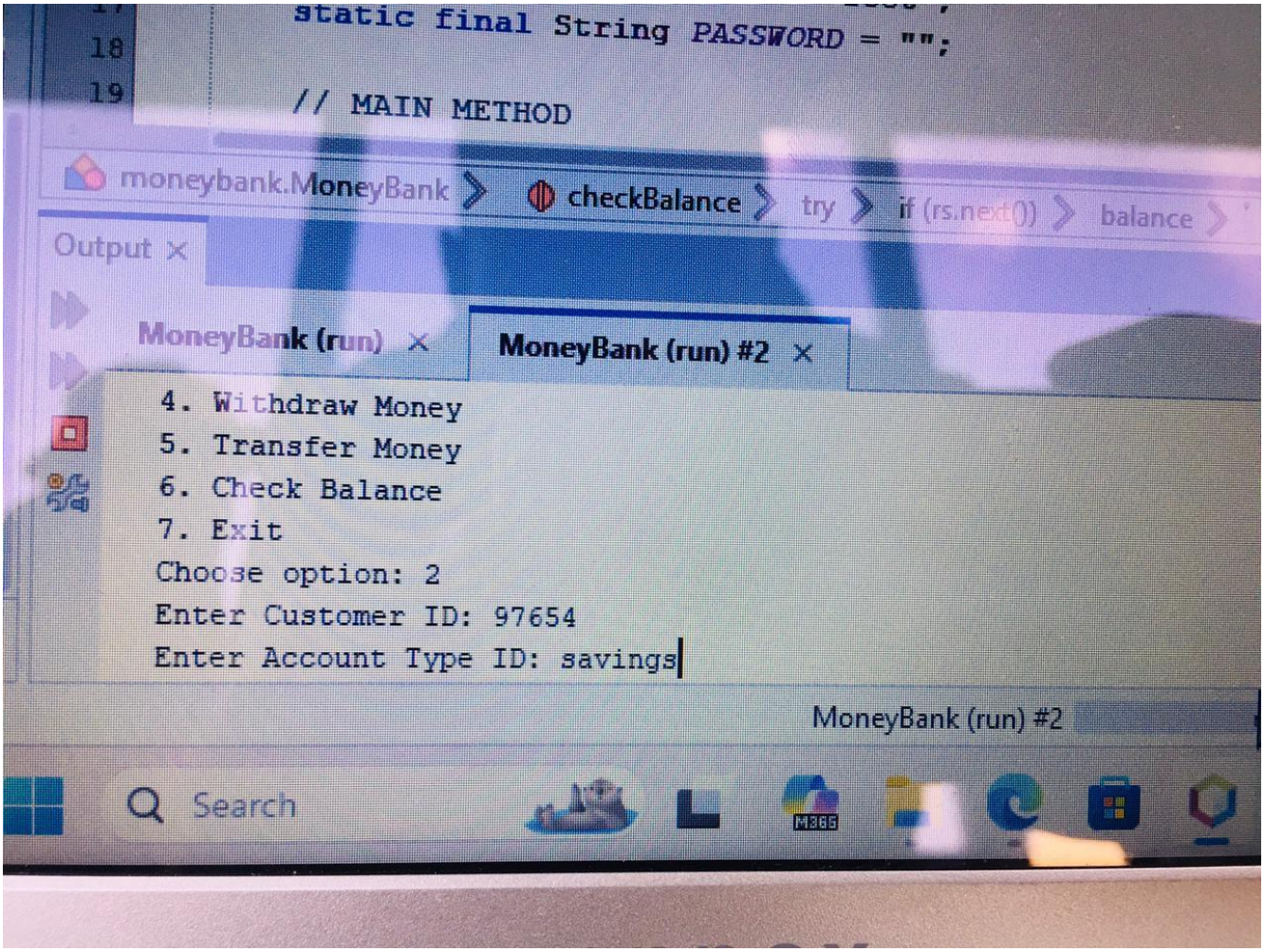
Main Menu

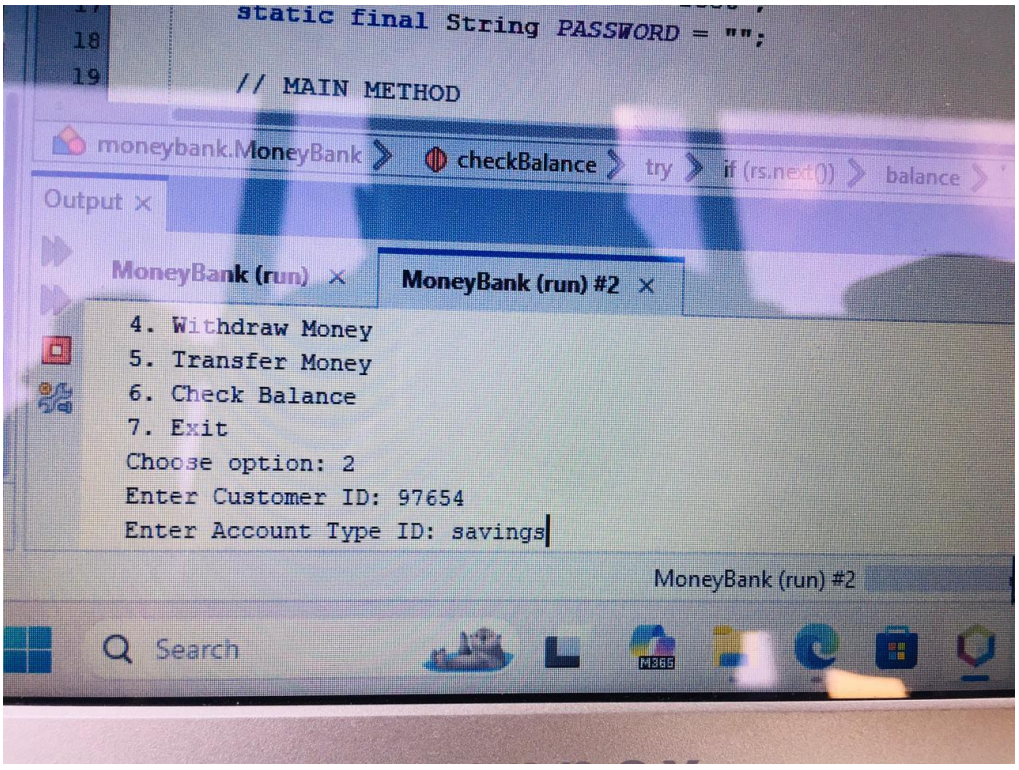


Customer Registration

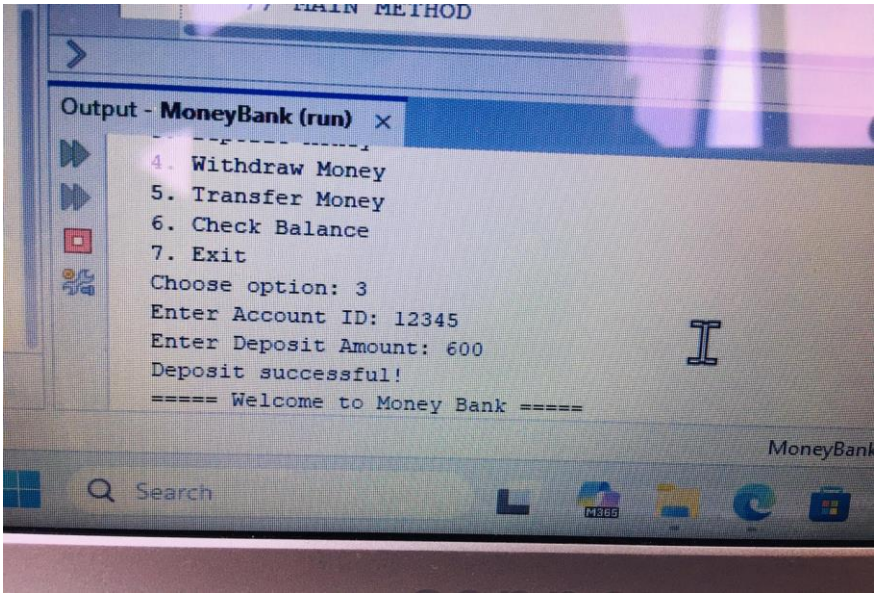


Account Creation

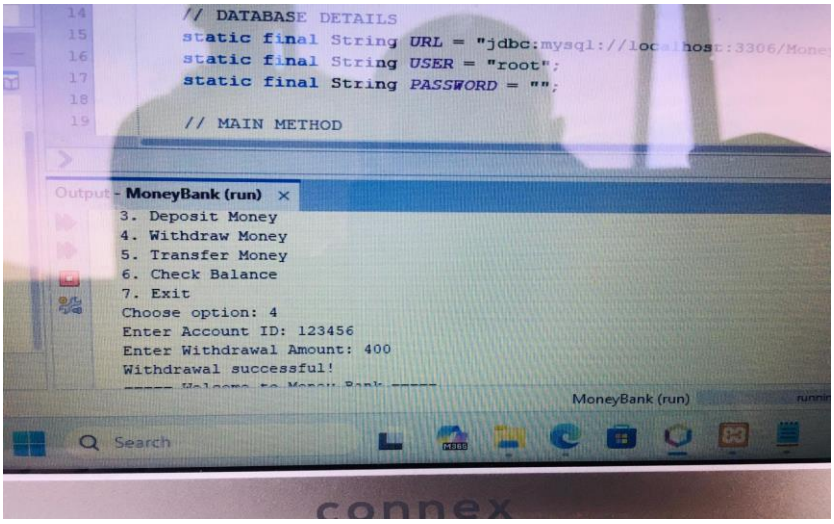

18 static final String PASSWORD = "";
19 // MAIN METHOD
moneybank.MoneyBank > checkBalance > try > if (rs.next()) > balance
Output
MoneyBank (run)
MoneyBank (run) #2
4. Withdraw Money
5. Transfer Money
6. Check Balance
7. Exit
Choose option: 2
Enter Customer ID: 97654
Enter Account Type ID: savings
MoneyBank (run) #2
Search


18 static final String PASSWORD = "";
19 // MAIN METHOD
moneybank.MoneyBank > checkBalance > try > if (rs.next()) > balance
Output
MoneyBank (run)
MoneyBank (run) #2
4. Withdraw Money
5. Transfer Money
6. Check Balance
7. Exit
Choose option: 2
Enter Customer ID: 97654
Enter Account Type ID: savings
MoneyBank (run) #2
Search

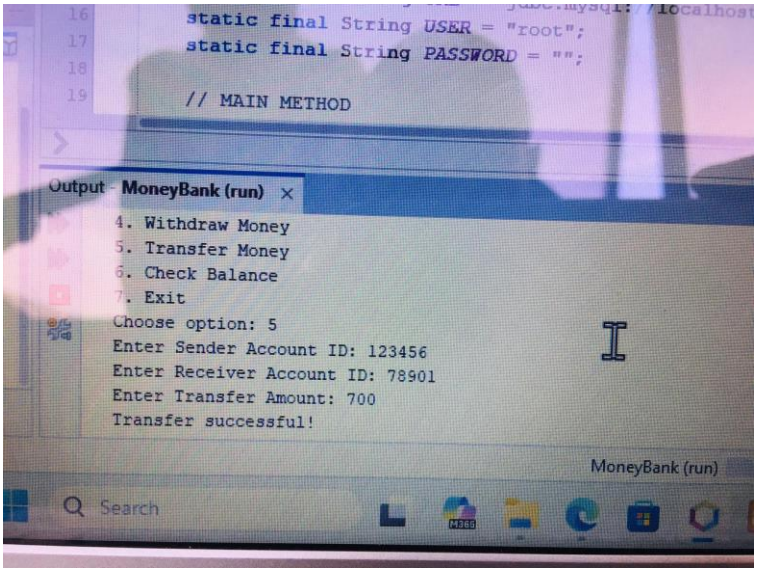
Deposit Feature



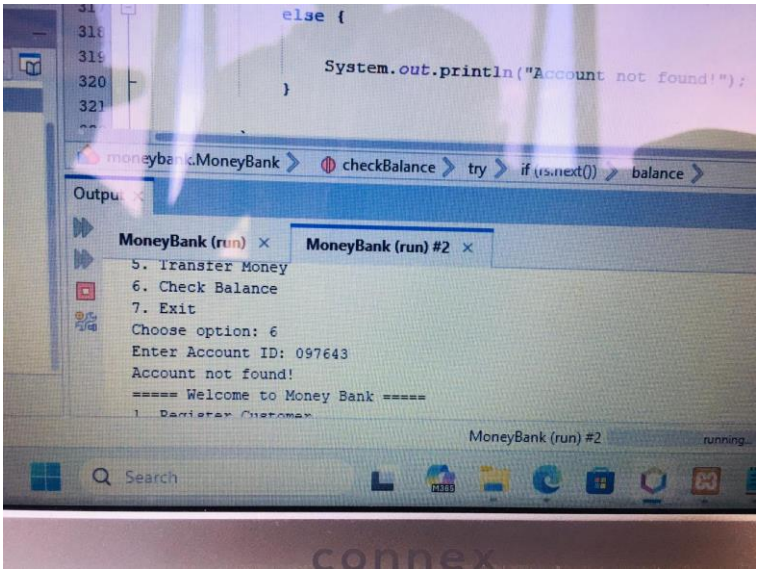
Withdrawal Feature



Transfer Feature



Balance Enquiry



4.3 Input and Output Descriptions

Feature	Input	Output
Register Customer	Name, phone, email	Customer created
Deposit	Account number, amount	Updated balance
Withdraw	Account number, amount	Withdrawal successful
Transfer	Source account, destination account, amount	Transfer successful

4.4 Error Handling

The system includes validation and exception handling.

Error	System Response
Invalid account number	Error message displayed
Insufficient balance	Withdrawal cancelled
Negative deposit amount	Transaction rejected
Database connection failure	Exception handled

5. USER MANUAL

5.1 How to Run the Application

1. Open the Java IDE
2. Import the project
3. Configure database connection settings
4. Run Main.java
5. Use the menu options

5.2 How to Set Up the Database

Step 1: Create Database

```
CREATE DATABASE banking_system;

USE banking_system;
```

Step 2: Create Tables

```
CREATE TABLE customer (
    customer_id INT AUTO_INCREMENT PRIMARY KEY,
    first_name VARCHAR(50),
    last_name VARCHAR(50),
    phone VARCHAR(20),
    email VARCHAR(100)
);


CREATE TABLE account (
    account_id INT AUTO_INCREMENT PRIMARY KEY,
    customer_id INT,
    account_type VARCHAR(20),
    balance DECIMAL(10,2),
    FOREIGN KEY (customer_id)
    REFERENCES customer(customer_id)
);
```

```
CREATE TABLE transaction (
    transaction_id INT AUTO_INCREMENT PRIMARY KEY,
    account_id INT,
    transaction_type VARCHAR(20),
    amount DECIMAL(10,2),
    transaction_date DATETIME,
    FOREIGN KEY (account_id)
    REFERENCES account(account_id)
);
```

5.3 Step-by-Step Instructions for Using the System

Registering a Customer

1. Launch the application
2. Select “Register Customer”
3. Enter customer information
4. Click submit

Depositing Money

1. Select “Deposit”
2. Enter account number
3. Enter amount
4. Confirm transaction

Transferring Money

1. Select “Transfer”
2. Enter source account
3. Enter destination account
4. Enter amount
5. Confirm transfer

5.4 Common Errors and Solutions

Problem	Solution
JDBC Driver Missing	Add MySQL Connector/J
Database Connection Failed	Verify username/password
SQL Syntax Error	Check SQL statements
NullPointerException	Initialize objects properly

6. CHALLENGES AND REFLECTIONS

6.1 Challenges Faced During Development

Challenge 1: Database Connectivity

The application initially failed to connect to MySQL due to incorrect JDBC configuration.

Challenge 2: Transfer Logic

Implementing transfers required updating two accounts simultaneously without data inconsistency.

6.2 How Challenges Were Resolved

Database Connectivity Solution

- Corrected JDBC URL
- Added MySQL JDBC driver

Transfer Logic Solution

- Used SQL transactions
- Added validation checks

6.3 Lessons Learned

Through this project, I learned:

- Java database connectivity using JDBC
- SQL database design
- Object-oriented programming
- Exception handling
- Transaction processing
- Database normalization

If given another opportunity, I would improve the GUI design and implement advanced security features.

6.4 Individual Contribution Statement

I was responsible for:

- Designing the database

- Implementing Java classes
- Creating transaction features
- Connecting the application to MySQL
- Testing and debugging the system

7. CONCLUSION

The Banking Management System successfully achieved its objectives by implementing core banking operations using Java and MySQL. The project demonstrated practical application of object-oriented programming, database design, and transaction management.

The system can be further improved by adding:

- User authentication
- Encryption
- Mobile banking features
- Advanced reporting

---

8. REFERENCES

1. Oracle Java Documentation
2. MySQL Documentation
3. JDBC API Documentation
4. Course notes and lecture materials